

模块 3：Planning — 规划的策略空间

模块概述

当 Agent 已经通过感知（Perception）获取了当前状态，通过目标分析明确了终点，接下来面对的核心问题就是：

如何将“从 A 到 B”的路径分解成可执行的步骤？

这就是 Planning 的本质。规划是连接“理解”与“行动”的桥梁——它把模糊的高层意图转化为具体的操作序列。

规划策略的选择空间比大多数人想象的要大。从最简单的线性任务列表，到树状分解、DAG 依赖图，再到多代理分工，每种策略都有其适用场景和代价。本模块将系统地探索这个策略空间，帮助你建立一个关于“何时用什么规划策略”的决策框架。

我们需要回答的关键问题：- 什么时候需要显式规划，什么时候隐式规划就够了？- 规划的粒度应该多细？- 当计划出错时，应该修补还是重来？- 多个 Agent 协作时，如何划分职责？

3.1 扁平任务列表

最简单的规划：列一个 TODO List

扁平任务列表是最朴素的规划形式——将任务分解为若干步骤，按顺序逐一执行。没有依赖关系的建模，没有并行的考量，就是一个线性的 checklist。

TODO：

1. 阅读报错日志，定位出错的文件
2. 读取该文件，理解上下文
3. 修改代码修复 bug
4. 运行测试验证
5. 提交代码

这看起来过于简单，但实际上是当前最主流 AI Coding Agent 的核心规划模式。

Claude Code 的隐式规划

Claude Code 在默认模式下并没有一个显式的“规划阶段”——它不会先输出一个结构化的计划文件再执行。相反，规划是**隐式地嵌入在模型的推理过程中的**。

当你给 Claude Code 一个任务（例如“修复这个 API 的 500 错误”），模型会在内部：

1. **隐式分析**：理解当前上下文（已打开的文件、错误信息、项目结构）
2. **隐式规划**：在生成下一个 action 之前，模型“想好”了大致的步骤
3. **逐步执行**：每次调用一个工具，根据结果决定下一步

这种模式的关键特征是**规划与执行交织在一起**。模型不会预先生成完整的步骤列表然后机械执行，而是在每一步根据最新信息动态决定下一步。

Claude Code 的内部推理（简化示意）

用户请求: "修复 login API 的 500 错误"

Turn 1:

思考: "我需要先看看错误日志或者相关代码"

行动: Grep("500", path="src/api/")

Turn 2:

思考: "找到了 src/api/auth.ts 有相关代码, 让我看看"

行动: Read("src/api/auth.ts")

Turn 3:

思考: "发现第 42 行 null check 缺失, 这就是问题"

行动: Edit("src/api/auth.ts", ...)

Turn 4:

思考: "修改完了, 需要验证"

行动: Bash("npm test")

优势

- **低开销**：没有额外的规划阶段，直接开始执行
- **自然适应**：每一步都基于最新的信息，天然具有适应性
- **简单可靠**：对于 80% 的日常任务（小 bug 修复、简单重构、添加小功能），这种方式足够好
- **减少浪费**：不会花时间制定一个后来证明无用的详细计划

劣势

- **缺乏全局视野**：容易陷入局部最优——修完一个文件才发现应该从另一个文件入手
- **无法并行**：步骤只能串行执行，浪费了可以并行的机会
- **对复杂任务力不从心**：当任务涉及 10+ 个文件、多个子系统时，没有显式规划容易迷失方向
- **难以回溯**：执行到一半发现方向错误时，没有明确的计划可以参照和修正

示例：一次简单的 Bug 修复

假设任务是“用户反馈登录页面的‘记住我’功能失效”。

一个扁平的隐式规划流程：

```
步骤 1：搜索 "remember me" 或 "rememberMe" 相关代码
→ 找到 src/components/LoginForm.tsx 和 src/api/auth.ts

步骤 2：阅读 LoginForm.tsx，检查前端逻辑
→ checkbox 状态正确传递给了 API

步骤 3：阅读 auth.ts，检查后端处理
→ 发现 cookie 的 maxAge 写成了 0 而不是 30 天

步骤 4：修复 auth.ts 中的 maxAge 值
→ 将 maxAge: 0 改为 maxAge: 30 * 24 * 60 * 60 * 1000

步骤 5：运行测试
→ 所有测试通过

步骤 6：提交
```

这个流程清晰、高效，不需要更复杂的规划策略。**大部分 Agent 任务都适合这种模式。**

何时扁平列表就够了

一个经验法则：如果任务满足以下条件，扁平任务列表就是最佳选择：

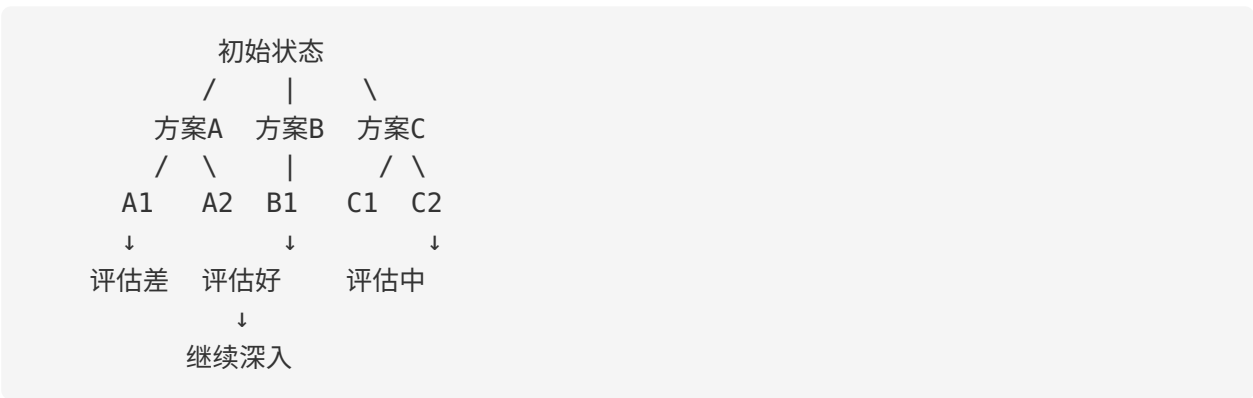
- 涉及的文件不超过 3-5 个
 - 没有需要权衡的多条路径
 - 不需要多个子任务并行
 - 失败的代价较低（可以轻松重试）
-

3.2 树状分解（Tree of Thoughts）

从链到树：当线性思考不够时

Chain-of-Thought（CoT）让 LLM 逐步推理，但它本质上是一条**单链**——模型沿着一条路径一直走下去。当问题有多个可行方案、或者需要在不确定中做出选择时，单链推理的局限性就暴露了。

Tree of Thoughts（Yao et al., 2023）提出了一种新范式：将推理过程组织成一棵树，每个节点是一个“思考状态”（thought state），可以从每个节点展开多个分支，通过评估选择最有前途的路径。



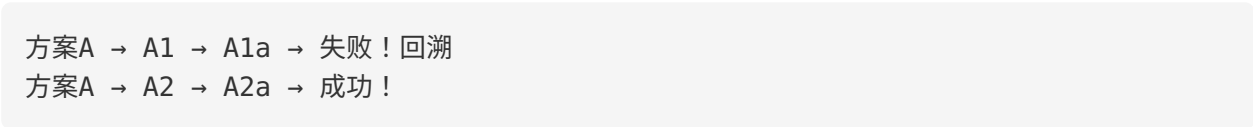
BFS vs DFS 探索策略

Tree of Thoughts 的核心是搜索策略的选择：

BFS（广度优先搜索）： - 先展开所有同层节点，评估后选择最好的一批继续 - 适合：方案之间差异较大，需要广泛比较的场景 - 代价：需要同时维护多个状态，token 消耗大



DFS（深度优先搜索）： - 沿一条路径深入，直到成功或确认失败再回溯 - 适合：路径之间相对独立，深入探索代价不高的场景 - 代价：可能在错误路径上浪费大量时间



每个节点的评估

树状分解的关键在于**评估函数**——如何判断一个中间状态的好坏：

1. **LLM 自评估**：让同一个模型判断“这个方案到目前为止是否有前途”
2. **投票机制**：生成多个评估，取多数意见
3. **外部验证**：运行测试、编译代码等，用客观结果评估
4. **启发式规则**：基于预定义的规则打分（如代码复杂度、修改范围等）

在 AI Coding Agent 的语境下，最实用的评估方式是**外部验证**——让 Agent 实际尝试一个方案，跑测试看结果。

何时树状分解有帮助

树状分解在以下场景特别有价值：

场景 1：多种可行的实现方案

任务：优化一个慢查询

方案 A（加索引）：

- 成本低，风险低
- 但可能效果有限

方案 B（重写查询逻辑）：

- 成本中，需要理解业务逻辑
- 可能带来显著提升

方案 C（引入缓存层）：

- 成本高，架构改动大
- 效果最好但复杂度高

场景 2：不确定的修复方向

任务：修复一个间歇性的 race condition

分支 1：怀疑是数据库连接池问题

- 检查连接池配置
- 加锁测试
- 结果：不是这个原因

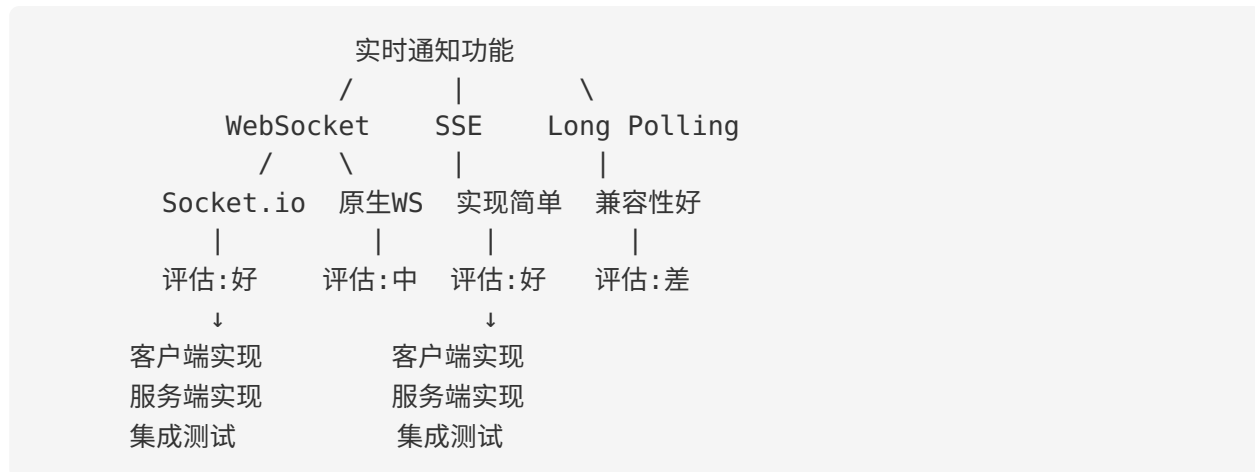
分支 2：怀疑是缓存过期时的并发读写

- 检查缓存逻辑

- 发现确实有问题
- 修复 & 验证

示例：功能实现的方案分解

假设任务是“为应用添加实时通知功能”：



Agent 可以先对三个方案做轻量级评估（检查现有技术栈、依赖、架构约束），然后选择最适合的方案深入实施。

实践中的局限

Tree of Thoughts 在论文中效果显著，但在工程实践中需要注意：

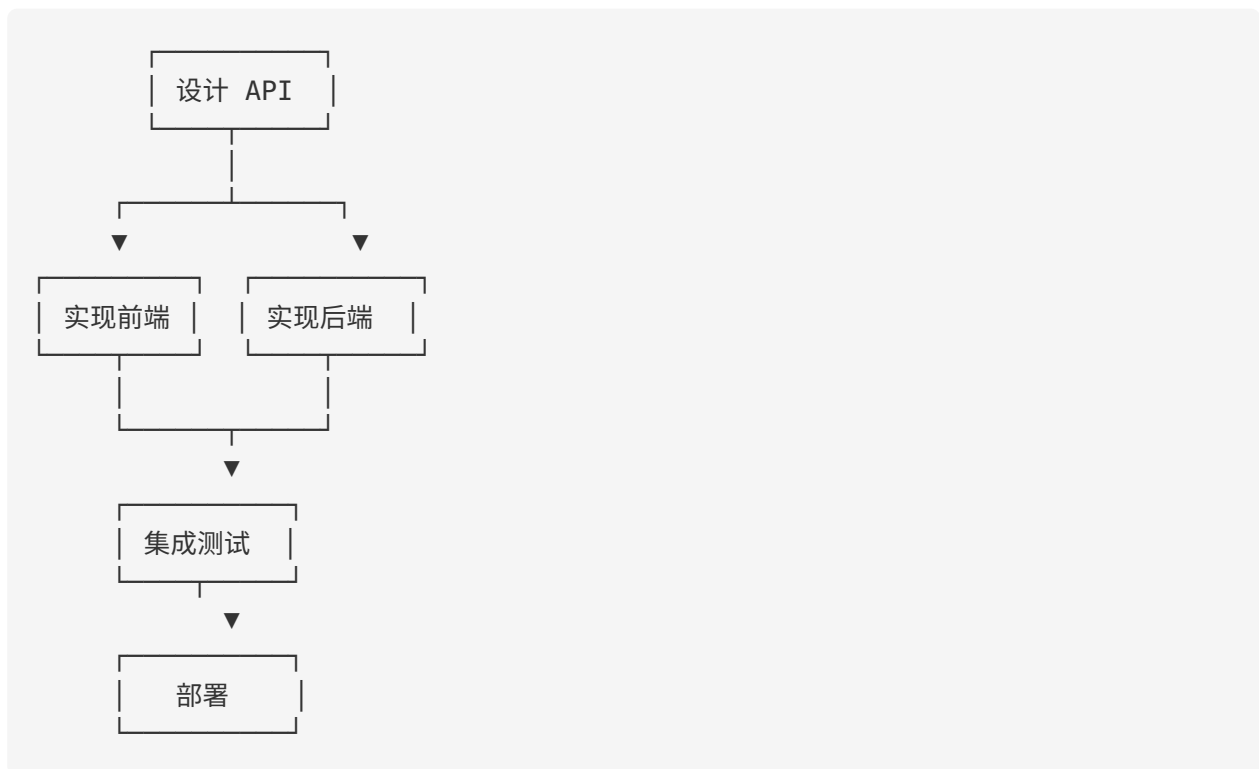
- **成本问题**：每个分支都需要 LLM 推理，token 消耗成倍增长
- **实现复杂度**：维护树状态、实现回溯机制的工程成本不低
- **简单任务得不偿失**：对于有明确解法的任务，树状搜索是浪费
- **评估瓶颈**：如果没有好的评估函数，展开再多分支也选不出最优

在实际的 Coding Agent 中，完整的 Tree of Thoughts 搜索较少直接使用。更常见的是一种**简化版**：Agent 在决策点考虑 2-3 个方案，快速评估后选一个执行，如果失败再回溯尝试另一个。这本质上是一种带回溯的 DFS。

3.3 DAG 依赖图

当任务之间存在依赖关系

现实世界的项目任务很少是完全独立的。实现功能 B 可能依赖功能 A 的 API；测试必须在实现之后；部署必须在测试通过之后。当这些依赖关系形成网络时，我们需要一种比线性列表更强大的表示方式——**有向无环图（DAG）**。



在这个例子中，前端和后端可以**并行**实现（因为它们只依赖 API 设计，互不依赖），而集成测试必须等两者都完成。

用 DAG 表示任务依赖

DAG 的形式化定义很直接：

- **节点**：每个节点代表一个任务
- **有向边**：如果任务 A → 任务 B，表示 B 依赖 A（A 必须在 B 之前完成）
- **无环**：不存在循环依赖（否则就死锁了）

伪代码：DAG 表示

```
tasks = {  
  "design_api":    { "deps": [] },  
  "impl_frontend": { "deps": ["design_api"] },  
  "impl_backend":  { "deps": ["design_api"] },  
}
```

```
"write_tests": { "deps": ["design_api"] },
"integration": { "deps": ["impl_frontend", "impl_backend",
"write_tests"] },
"deploy": { "deps": ["integration"] },
}
```

拓扑排序与执行

有了 DAG，我们通过**拓扑排序（Topological Sort）**得到一个合法的执行顺序——保证每个任务在其所有依赖完成后才执行。

拓扑排序结果（多种合法顺序之一）：

1. design_api
2. impl_frontend, impl_backend, write_tests ← 可并行
3. integration
4. deploy

关键洞察：拓扑排序不是唯一的——所有没有前驱依赖的节点可以**同时**启动。这就是并行化的来源。

识别可并行的任务

在 DAG 中，任务的并行性由其依赖结构决定：

规则：两个任务可以并行，当且仅当它们之间没有（直接或间接的）依赖关系。

等价描述：在 DAG 中，两个节点可以并行执行，当且仅当它们之间不存在有向路径（无论方向）。

实际操作中，可以按“层次”组织执行：

```
Layer 0: [design_api] → 串行执行
Layer 1: [impl_frontend, impl_backend, write_tests] → 并行执行
Layer 2: [integration] → 串行执行
Layer 3: [deploy] → 串行执行
```

每一层内的任务可以并行，层与层之间串行。这就是一个简单但有效的调度策略。

Claude Code 的 Agent Teams 与依赖管理

Claude Code 通过 Agent Teams 和 `TaskCreate / TaskUpdate` API 提供了一套实用的 DAG 依赖管理机制：

创建任务并设置依赖关系

```
TaskCreate: "设计 API 接口"           → task_id: "1"
TaskCreate: "实现前端组件"           → task_id: "2"
TaskCreate: "实现后端服务"           → task_id: "3"
TaskCreate: "编写集成测试"           → task_id: "4"
```

设置依赖

```
TaskUpdate: task_id="2", addBlockedBy=["1"]    # 前端依赖 API 设计
TaskUpdate: task_id="3", addBlockedBy=["1"]    # 后端依赖 API 设计
TaskUpdate: task_id="4", addBlockedBy=["2","3"] # 测试依赖前后端
```

Team Lead 可以创建多个 Agent，将不同任务分配给不同的 Agent 执行：

Team Lead 的协调流程

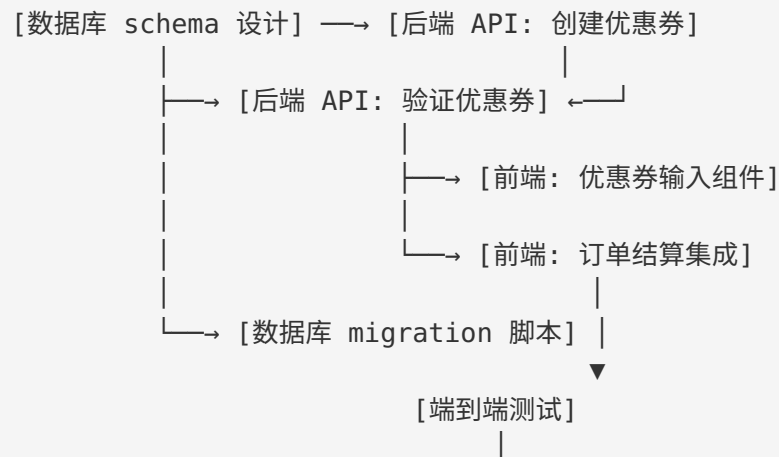
1. 创建任务和依赖关系
2. 自己执行或分配 task "1" (设计 API)
3. task "1" 完成后，将 task "2" 分配给 Agent-A，task "3" 分配给 Agent-B
4. Agent-A 和 Agent-B 并行工作
5. 两者都完成后，执行 task "4" (集成测试)

这种方式的优势在于： - **显式依赖**：通过 `blockedBy` 字段明确表达任务间的前后关系 - **并行执行**：无依赖的任务可以同时分配给不同 Agent - **状态可追踪**：每个任务有 `pending / in_progress / completed` 状态

示例：一个有依赖的多组件功能

假设任务是“为电商应用添加优惠券功能”：

DAG 结构：



执行调度：

Phase 1（串行）：

- 数据库 schema 设计

Phase 2（并行）：

- 后端 API：创建优惠券
- 数据库 migration 脚本

Phase 3（串行，等 Phase 2 完成）：

- 后端 API：验证优惠券

Phase 4（并行）：

- 前端：优惠券输入组件
- 前端：订单结算集成

Phase 5（串行，等 Phase 4 完成）：

- 端到端测试

Phase 6：

- 文档更新

通过 DAG 分析，我们将一个原本可能需要 7 个串行步骤的任务，优化为 6 个阶段（其中两个阶段包含并行任务），显著缩短了总耗时。

DAG 规划的挑战

- **依赖识别**：让 LLM 准确识别任务间的依赖关系并不总是可靠的
 - **粒度选择**：任务切分太细，依赖关系变得复杂；太粗，并行度不够
 - **动态依赖**：有些依赖只有在执行过程中才能发现（如“需要先安装一个之前没预料到的库”）
 - **失败传播**：DAG 中一个节点失败可能阻塞整个下游子图
-

3.4 多代理分工

为什么需要多个 Agent?

单个 Agent 处理复杂任务面临几个核心瓶颈：

1. **Context 窗口限制**：一个大型功能可能涉及几十个文件，单个 Agent 的 context 不够装
2. **串行执行瓶颈**：一个 Agent 同一时间只能做一件事
3. **专业化需求**：不同子任务可能需要不同的“思维模式”（设计 vs 编码 vs 测试）

多代理系统通过“分工”来解决这些问题。但分工的方式有很大差异，我们来看几种代表性的范式。

MetaGPT：基于角色的流水线

MetaGPT (Hong et al., 2023) 借鉴了真实软件公司的组织结构，定义了明确的角色分工：

Product Manager → Architect → Engineer → QA

产品经理：将需求转化为 PRD（产品需求文档）

↓

架构师：将 PRD 转化为系统设计和接口定义

↓

工程师：根据设计实现代码

↓

QA：编写测试、验证实现

核心理念：

- **结构化中间产物** (Structured Intermediate Outputs)：每个角色不只是产出文本，而是产出结构化的工件——PRD、设计文档、UML 图、代码、测试报告
- **角色约束**：每个 Agent 只能执行其角色范围内的操作
- **SOP (标准操作流程)**：每个角色有预定义的工作流程

MetaGPT 风格的角色定义 (简化)

```
class ProductManager(Role):  
    actions = [WritePRD, ReviewRequirements]  
    watch = [UserRequirement] # 监听用户需求
```

```
class Architect(Role):
```

```
actions = [WriteDesign, WriteAPISpec]
watch = [PRD] # 监听 PRD 产出

class Engineer(Role):
    actions = [WriteCode, FixBug]
    watch = [Design, APISpec] # 监听设计产出
```

优势： - 角色边界清晰，减少了 Agent 之间的冲突 - 结构化产物提供了明确的交接点 - 模拟了被证明有效的人类工程组织

劣势： - 角色划分过于刚性——现实中经常需要跨角色协作 - 流水线模型假设需求是线性传递的，不适合需要频繁反馈的场景 - 中间产物的格式需要预定义，灵活性受限

AutoGen：对话式协作

AutoGen (Wu et al., 2023) 采取了完全不同的方式——Agent 之间通过**对话**协作：

```
Agent-A (Coder): "我实现了用户认证模块，代码如下..."
Agent-B (Reviewer): "我发现了一个安全问题：密码没有加盐哈希"
Agent-A (Coder): "好的，让我修复..."
Agent-C (Tester): "修复后的版本通过了所有安全测试"
```

核心理念：

- **对话即协调：** Agent 之间不需要预定义的工作流，通过自然语言对话来协调
- **灵活的 Agent 组合：** 可以根据任务动态组合不同的 Agent
- **Human-in-the-loop：** 人类可以作为对话的一方随时参与

```
# AutoGen 风格的多代理对话（简化）
assistant = AssistantAgent("coder", llm_config=...)
reviewer = AssistantAgent("reviewer", llm_config=...)
user_proxy = UserProxyAgent("user", human_input_mode="TERMINATE")

groupchat = GroupChat(
    agents=[user_proxy, assistant, reviewer],
    messages=[],
    max_round=20,
)
```

优势： - 高度灵活，不需要预定义流程 - 适合探索性任务（不确定最佳路径时） - 容易引入人类反馈

劣势： - 对话可能发散——两个 Agent 可能陷入无意义的来回 - 缺乏结构化产物，中间结果难以追踪 - Token 消耗大（每个 Agent 都需要看完整对话历史）

ChatDev：模拟软件公司

ChatDev 更激进地模拟了一个软件公司的完整运作：

CEO ↔ CTO ↔ Programmer ↔ Tester ↔ Art Designer

阶段 1（设计）：CEO 和 CTO 讨论产品方向

阶段 2（编码）：CTO 和 Programmer 讨论实现

阶段 3（测试）：Programmer 和 Tester 讨论测试

阶段 4（文档）：整理文档和最终产出

ChatDev 的特点是**两两对话**（chat chain）——每个阶段只有两个角色参与对话，降低了多方对话的复杂性。

Claude Code Agent Teams：实用的多代理协调

与上述学术系统相比，Claude Code 的 Agent Teams 更偏向工程实用：

Team Lead（主 Agent）

├─ Agent-A（通过 Agent tool 创建，处理子任务 1）

├─ Agent-B（通过 Agent tool 创建，处理子任务 2）

└─ Agent-C（通过 Agent tool 创建，处理子任务 3）

核心机制：

1. **Team Lead 统筹：**主 Agent 负责创建任务、分配工作、协调进度
2. **Task 系统：**通过 `TaskCreate` / `TaskUpdate` / `TaskList` 管理任务状态和依赖
3. **消息传递：**通过 `SendMessage` 进行 Agent 间通信
4. **隔离执行：**每个子 Agent 可以在独立的 worktree 中工作，避免文件冲突

Claude Code Agent Teams 工作流示例

1. Team Lead 分析任务，创建 TaskList

2. Team Lead 创建子 Agent：

Agent("frontend-dev", prompt="实现前端组件...", team_name="feature-x")

Agent("backend-dev", prompt="实现后端 API...", team_name="feature-x")

3. 子 Agent 各自工作：

- frontend-dev：读取任务 → 实现 → 标记完成

- backend-dev：读取任务 → 实现 → 标记完成

4. Team Lead 收到完成通知，执行集成测试
5. Team Lead 通知子 Agent shutdown

与学术系统的关键区别：

特性	MetaGPT	AutoGen	Claude Code Teams
角色定义	固定角色	灵活角色	灵活，但由 Lead 分配
协调方式	SOP 流水线	自由对话	Task 系统 + 消息
隔离性	共享状态	共享对话	Worktree 隔离
实用性	学术原型	框架级	生产级工具

Trade-offs：角色刚性 vs 协调开销

多代理系统的核心 trade-off：

更多结构化（MetaGPT 方向）： - □减少冲突和重复工作 - □更可预测的行为 - □灵活性差，难以应对意外情况 - □角色间的交接成本

更多灵活性（AutoGen 方向）： - □适应性强 - □容易处理模糊或探索性任务 - □协调开销大，对话容易发散 - □结果不可预测

实用建议： - 对于结构清晰、流程明确的任务（如“实现这个 spec”），偏向 MetaGPT 式的结构化分工 - 对于探索性任务（如“调查并修复这个性能问题”），偏向 AutoGen 式的灵活协作 - Claude Code Agent Teams 处于两者中间：通过 Task 系统提供结构，通过消息传递提供灵活性

3.5 Plan 的粒度

粒度之争：要多细？

规划的粒度是一个经常被低估的设计决策。粒度不当会带来严重问题——太细则僵化浪费，太粗则缺乏指导意义。

细粒度规划：每一步都提前指定

细粒度计划示例

1. 打开 `src/models/user.ts`
 2. 在第 15 行后添加 ``couponCode: string`` 字段
 3. 打开 `src/api/orders.ts`
 4. 在 `createOrder` 函数的第 42 行添加优惠券验证逻辑
 5. 验证逻辑：调用 `validateCoupon(code)`，如果无效返回 400
 6. 打开 `src/services/coupon.ts`（新建文件）
 7. 实现 `validateCoupon` 函数...
- （以下还有 20+ 步）

优点： - 执行时不需要决策，机械执行即可 - 容易追踪进度 - 适合对 Agent 行为做严格约束

缺点： - **脆弱性高：**如果第 15 行的实际内容与预期不符，整个计划就废了 - **信息浪费：**在规划阶段花大量 token 描述细节，但这些细节可能在执行时才最准确 - **无法应对意外：**计划越详细，遇到意外时越难调整

粗粒度规划：只定义高层目标

粗粒度计划示例

1. 设计优惠券功能的数据模型
2. 实现后端 API
3. 实现前端组件
4. 编写测试
5. 集成验证

优点： - 规划成本低 - 对意外情况有弹性——每一步的具体实现可以根据实际情况决定 - 不会因为细节预判错误而导致计划失败

缺点： - 每一步仍需要 Agent 做大量决策 - 对复杂步骤缺乏指导，Agent 可能在某一步迷失 - 难以准确分配给不同 Agent（粒度太粗，多个 Agent 的工作范围可能重叠）

规划视野问题（Planning Horizon）

规划视野指的是**计划应该看多远**。这与粒度密切相关但不完全相同：

短视野 + 细粒度："接下来 3 步要做什么"（每步很具体）

短视野 + 粗粒度："接下来要做什么大方向"

长视野 + 粗粒度: "整个功能的实现路径" (每步较模糊)
长视野 + 细粒度: "整个功能每一步的详细操作" (很少可行)

关键洞察: 规划的准确度随视野增长而急剧下降。当前最有效的做法是:

对近期任务细粒度规划, 对远期任务粗粒度规划。随着执行推进, 逐步细化。

这就引出了自适应粒度的概念。

自适应粒度: 从粗到细

最实用的策略是**渐进式细化 (Progressive Refinement)**:

阶段 1 - 粗规划:

"实现优惠券功能: 需要数据模型、API、前端、测试"

阶段 2 - 当前阶段细化:

"数据模型设计:

- Coupon 表: id, code, discount_type, discount_value, expiry, ...
- 与 Order 表的关联方式
- Migration 脚本"

阶段 3 - 执行中进一步细化:

"Migration 脚本:

- 创建 coupons 表
- 在 orders 表添加 coupon_id 外键
- 添加索引"

每次只细化即将执行的部分, 保持远期计划的模糊性。这样既有全局方向, 又不会因为过早确定细节而产生浪费。

经验证据: 规划何时有帮助, 何时有害?

来自学术研究和工程实践的观察:

规划有帮助的情况: - 任务有明确的结构和可分解性 - 多个 Agent 需要协作 (需要明确职责边界) - 任务成本高 (如需要大量修改), 试错代价大 - 用户需要审查和批准方案 (Plan Mode)

规划可能有害的情况: - 任务简单到不需要规划 (过度规划浪费 token 和时间) - 环境高度不确定 (计划很快就会过时) - Agent 的隐式规划已经足够好 (如 Claude Code 的默认模式处理小任务)

一个实用的判断标准：

```
if 任务步骤 <= 5 且 不需要多 Agent:
    不需要显式规划
elif 任务步骤 <= 15 且 依赖关系简单:
    粗粒度规划 + 渐进细化
else:
    DAG 级别的详细规划 + 多 Agent 分工
```

来自 SWE-bench 等基准测试的教训：

在代码修复基准上，最好的 Agent 系统往往不是规划最详尽的，而是在**规划和执行之间找到了好的平衡**的系统。过度规划的系统花太多 token 在计划上，留给实际执行的 token 不够；而完全不规划的系统在复杂任务上容易迷失方向。

3.6 动态重规划

计划赶不上变化

无论规划多周密，执行中总会遇到意外：

- 修改文件 A 时发现它的接口已经变了
- 测试失败暴露了一个之前不知道的依赖关系
- 用户在中途改变了需求
- 某个库的 API 与文档不一致

动态重规划 (Dynamic Replanning) 是 Agent 在发现计划偏离预期时调整策略的能力。这是区分“机械执行脚本”和“智能 Agent”的关键能力。

检测计划偏离

Agent 需要识别“计划出错了”的信号：

硬信号（明确的失败）： - 测试失败 - 编译错误 - 运行时异常 - 工具调用返回错误

软信号（可能的偏离）： - 读到的文件内容与预期不同 - 发现了计划中没有考虑的依赖 - 某一步耗时远超预期 - 中间结果的质量不够好

```
# 偏离检测的伪代码
for step in plan:
    result = execute(step)
```

```
if result.is_hard_failure():
    trigger_replan("硬失败", step, result)

elif result.diverges_from_expectation():
    if divergence_is_significant(result):
        trigger_replan("显著偏离", step, result)
    else:
        log_and_continue(result)
```

重规划策略

检测到偏离后，有三种主要的应对策略：

策略 1：修补计划（Patch the Plan）

在当前计划的基础上做局部调整，保持整体不变。

原计划：A → B → C → D → E

B 步骤失败

修补后：A → B' → C → D → E （用 B' 替换 B）

或者：A → B1 → B2 → C → D → E （拆分 B 为更小的步骤）

适用场景：失败是局部的，不影响后续步骤的逻辑。

例如：修改某个文件时发现函数签名变了，只需要调整当前步骤的修改方式，后续步骤的逻辑不受影响。

策略 2：从头重新规划（Replan from Scratch）

丢弃当前计划，基于新的理解重新制定方案。

原计划：A → B → C → D → E

发现原计划的基本假设错了

新计划：X → Y → Z （完全不同的方案）

适用场景：发现了颠覆性的新信息，原有计划的基础不再成立。

例如：原计划是修改现有 API，但发现该 API 已被废弃，需要用全新的 API 重新实现。

策略 3：升级处理 (Escalate)

将问题报告给上一级（用户或 Team Lead），请求指导。

Agent: "我在执行步骤 3 时发现了一个问题：
原计划假设数据库是 PostgreSQL，
但实际上项目使用的是 MongoDB。
这影响了后续所有数据库操作的实现方式。
请指示：
(a) 切换到 MongoDB 实现
(b) 添加 PostgreSQL 支持
(c) 使用 ORM 抽象层"

适用场景： - Agent 无法自行判断最佳方案 - 决策涉及架构层面的权衡 - 继续执行的成本较高

Claude Code 的重规划方式

Claude Code 的重规划是**隐式且持续的**。因为它没有显式的计划文件，所以“重规划”就是在下一个 turn 里，模型根据最新信息（包括失败信息）重新推理该怎么做。

Turn 1: 模型决定修改 auth.ts
Turn 2: 修改后运行测试 → 测试失败
Turn 3: 模型看到失败信息，重新分析问题
→ "原来是 session 模块也需要修改"
Turn 4: 修改 session.ts
Turn 5: 再次运行测试 → 通过

这种方式的优雅之处在于：**每个 turn 都是一次隐式的重规划**。模型在每次行动前都会基于所有已知信息（包括之前的失败）重新评估下一步最优行动。

但这也有局限性：

- **没有历史记录：** 不像显式计划那样可以回顾“我们偏离了什么”
- **可能陷入循环：** 没有显式计划作为锚点，模型可能反复尝试同一种方法
- **对长期任务不利：** context 窗口有限，早期的失败信息可能被压缩掉

重规划的成本分析

重规划不是免费的。我们需要权衡：

重规划成本 = 规划时间 + 已完成工作的浪费 + 新方向的切换成本

vs

坚持坏计划的成本 = 每步递增的修复成本 + 最终返工成本

一般来说，**越早发现偏离、越早重规划越好**。随着执行的深入，沉没成本增加，重规划的心理和实际障碍都在增大。

这就是为什么好的 Agent 系统需要**频繁验证点**：

计划：step1 → 验证 → step2 → 验证 → step3 → 验证 → ...

而不是：

计划：step1 → step2 → step3 → ... → step10 → 验证

在 Coding Agent 中，“验证”最常见的形式是**运行测试**。每次修改后运行测试，就是一个自然的偏离检测点。

关键论文导读

Tree of Thoughts (Yao et al., 2023)

论文: Tree of Thoughts: Deliberate Problem Solving with Large Language Models

核心贡献： - 将 LLM 的推理过程从线性链（Chain-of-Thought）扩展为树状结构 - 提出了通用的 Tree of Thoughts 框架：thought decomposition + thought generator + state evaluator + search algorithm - 在需要前瞻和回溯的任务上（如 Game of 24、Crossword）显著优于 CoT

关键设计： - **Thought decomposition**：将问题分解为若干步骤，每步产生一个“thought” - **Thought generator**：在每个节点生成 k 个候选 thought（采样或提议） - **State evaluator**：用 LLM 评估每个状态（投票或打分） - **Search algorithm**：BFS 或 DFS 遍历思考树

对 Agent 设计的启示： - 在关键决策点不要只考虑一条路径 - 评估能力与生成能力同样重要 - 搜索策略的选择取决于问题结构 - 但要注意成本——完整的树搜索在实际应用中可能过于昂贵

MetaGPT (Hong et al., 2023)

论文: MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework

核心贡献: - 提出将人类软件工程实践 (SOP) 编码为多 Agent 协作框架 - 引入结构化中间产物 (如 PRD、设计文档、API spec) 作为 Agent 间的通信协议 - 通过角色化减少了多 Agent 系统中常见的信息过载和幻觉问题

关键设计: - **Role-based Agent:** 每个 Agent 有明确角色、观察范围和行动空间 -

Structured Communication: Agent 之间通过文档 (而非自由对话) 沟通 -

Subscription Mechanism: Agent 只接收与自己角色相关的信息 - **Executable**

Feedback: 通过代码执行提供客观反馈

对 Agent 设计的启示: - 结构化通信比自由对话更高效、更可控 - 角色约束能有效减少幻觉和跑题 - 人类已验证的工作流程是好的起点

AutoGen (Wu et al., 2023)

论文: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation

核心贡献: - 提出了可定制的 Agent 对话框架 - 统一了多种多 Agent 对话模式 (两两对话、群聊、层级对话等) - 支持灵活的 Human-in-the-Loop 集成

关键设计: - **ConversableAgent:** 所有 Agent 的基类, 支持发送/接收消息 -

AssistantAgent: LLM-powered Agent, 可配置 system prompt - **UserProxyAgent:** 代理用户的 Agent, 可执行代码、请求人类输入 - **GroupChat:** 多 Agent 群聊协调, 支持多种发言策略

对 Agent 设计的启示: - 对话是一种通用且灵活的协调机制 - Agent 的可组合性 (composability) 是框架设计的关键 - 不同任务适合不同的对话拓扑

实操环节

Claude Code Plan Mode 实战

Claude Code 的 Plan Mode 让 Agent 先制定方案、获得用户批准后再执行。这是显式规划在实际工具中的体现。

练习 1：使用 Plan Mode 重构代码

目标：体验“先规划、再执行”的工作流。

```
# 启动 Claude Code
claude

# 进入 Plan Mode (两种方式)
# 方式 1：在对话中使用 Shift+Tab 切换到 Plan 模式
# 方式 2：Agent 自动进入 Plan Mode (对于复杂任务)
```

任务示例：

用户："请将 src/utils/ 目录下的所有函数从 CommonJS 迁移到 ES Modules"

Plan Mode 下的 Agent 行为：

1. 探索 src/utils/ 目录结构
2. 识别所有需要修改的文件
3. 分析依赖关系（哪些文件被其他文件 import）
4. 制定修改计划：
 - 按依赖关系的拓扑顺序修改
 - 先修改没有依赖的叶子模块
 - 再修改依赖它们的上层模块
5. 将计划写入 plan 文件
6. 调用 ExitPlanMode 请求用户审批

用户审批后：

Agent 按计划逐步执行修改

练习 2：对比有无规划的效果

实验设计：

任务："为项目添加国际化（i18n）支持"

实验 A - 无显式规划：

直接让 Claude Code 开始工作
观察：它会怎么做？会遗漏什么？

实验 B - 使用 Plan Mode：

先让 Claude Code 制定方案
审查方案中是否考虑了：

- 目标语言选择
- 翻译文件格式

- 已有代码的修改范围
 - 运行时语言切换机制
- 批准后执行

对比：

- 总耗时
- 最终质量
- 是否有返工

练习 3：Agent Teams 并行规划与执行

任务：用 Agent Teams 实现一个完整功能。

工作流：

步骤 1：Team Lead 分析任务，创建 DAG

步骤 2：创建 Team 并分配子 Agent

- Agent "api-dev"：负责后端 API 实现
- Agent "ui-dev"：负责前端 UI 实现
- Agent "test-dev"：负责测试编写

步骤 3：通过 TaskCreate 创建任务并设置依赖

- | | |
|---------------------|-----------------------------|
| Task 1: "设计 API 接口" | → 无依赖 |
| Task 2: "实现后端 API" | → blocked by Task 1 |
| Task 3: "实现前端 UI" | → blocked by Task 1 |
| Task 4: "编写 API 测试" | → blocked by Task 2 |
| Task 5: "编写 UI 测试" | → blocked by Task 3 |
| Task 6: "端到端测试" | → blocked by Task 4, Task 5 |

步骤 4：执行

- Team Lead 完成 Task 1
- 将 Task 2 分配给 api-dev, Task 3 分配给 ui-dev (并行)
- 完成后分配测试任务
- 最后执行集成测试

步骤 5：复盘

- 并行化节省了多少时间？
- Agent 之间是否有冲突？
- 任务粒度是否合适？

练习 4：动态重规划

任务：故意制造一个需要重规划的场景。

步骤 1: 让 Agent 制定一个计划来修改某个模块

步骤 2: 在 Agent 执行到一半时, 手动修改一个关键文件 (模拟"别人改了代码")

步骤 3: 观察 Agent 如何处理意外情况:

- 它发现了偏离吗?
- 它是修补计划还是重做?
- 它是否请求了用户确认?

本模块小结

核心知识点回顾

本模块探索了 Agent 规划的策略空间，从最简单到最复杂：

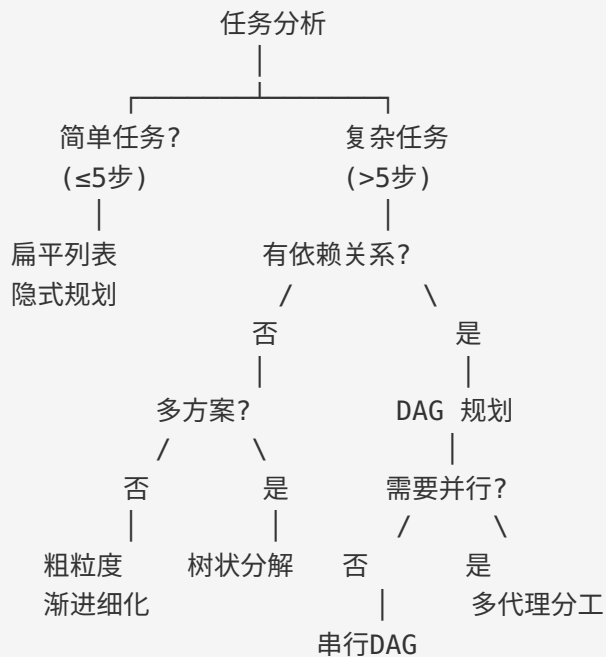
策略	复杂度	适用场景	代表系统
扁平任务列表	低	简单任务，步骤 ≤ 5	Claude Code 默认模式
树状分解	中	多方案评估，不确定性高	Tree of Thoughts
DAG 依赖图	中-高	有依赖关系的多步任务	Claude Code Agent Teams
多代理分工	高	大型任务，需要并行化	MetaGPT / AutoGen

规划不是越多越好——关键是找到**规划投入与执行效率的平衡点**。

关键 Take-aways

- 大多数任务不需要复杂规划：** 扁平列表 + 隐式规划就能解决 80% 的日常任务
- 规划粒度应该是自适应的：** 近期细、远期粗，边执行边细化
- 动态重规划是必需的能力：** 没有任何计划能完美预见所有情况，频繁的验证点（特别是运行测试）是早期发现偏离的关键
- 多 Agent 的价值在于并行化和专业化：** 但协调开销是真实存在的，只有任务足够大时才值得
- 结构化中间产物优于自由文本：** MetaGPT 的核心贡献之一就是证明了这一点——Agent 之间用文档而非对话沟通更高效
- Tree of Thoughts 在工程中的价值是思维方式：** 完整的树搜索成本太高，但“在关键决策点考虑多个方案”的思维方式非常有价值

规划策略选择决策树



思考题

- 规划 vs 执行的资源分配：**如果你有固定的 token 预算（例如 100k tokens），你会把多少比例分配给规划、多少分配给执行？这个比例应该随任务复杂度如何变化？
- 隐式 vs 显式规划：**Claude Code 默认使用隐式规划（模型在推理中隐含地规划），而 Plan Mode 是显式规划。在你的工程实践中，哪种方式的效果更好？能否设计一种介于两者之间的规划机制？
- 多 Agent 的最小必要人数：**对于一个中等复杂度的功能（如“为 Web 应用添加评论系统”），你认为最少需要几个 Agent？最佳数量是多少？增加 Agent 数量的边际收益在什么时候开始递减？
- 重规划的触发条件：**设计一套具体的规则来判断何时应该修补计划、何时应该彻底重新规划。考虑以下维度：已完成进度、失败的严重程度、剩余计划的可靠性。
- Tree of Thoughts 的工程化：**如果你要在一个 Coding Agent 中实现 Tree of Thoughts，你会选择在哪些决策点展开树搜索？如何控制搜索的成本？你的评估函数会是什么？
- 规划粒度的自动选择：**能否设计一个机制，让 Agent 自动决定当前任务需要多细的规划？这个机制需要考虑哪些因素（任务复杂度、不确定性、成本约束等）？

7. **跨项目的规划模式迁移**：一个 Agent 在项目 A 上学到的“这种类型的任务最好用 DAG 规划”的经验，能否迁移到项目 B？如何实现这种迁移？

下一模块预告：模块 4 将探讨 Execution & Tool Use——当计划制定好之后，Agent 如何实际执行操作、调用工具、处理结果。规划是“想”，执行是“做”，两者的配合决定了 Agent 的最终效果。